

Лабораторная работа № 8.

Миграции и начальное заполнение базы данных

Тема: Prisma Migrate и seeding базы данных Internet Shop

Цель: Научиться создавать и применять миграции Prisma, контролировать историю изменений схемы и воспроизводимо заполнять development database начальными данными.

Дисциплина: Node.js и Express.js

Исходные условия

Продолжайте работу с проектом `express-shop`. В проекте уже используются PostgreSQL, Prisma ORM и модели `Category` и `Product`.

В этой работе необходимо изменить структуру `Product` через Prisma Migrate, проверить созданный SQL, а затем настроить воспроизводимое начальное заполнение базы данных.

Перед началом

Работа выполняется только с development database. Не выполняйте `prisma migrate reset` для базы данных с ценными или production-данными.

1. Проверьте текущее состояние миграций

PowerShell

```
npx prisma@7 migrate status
```

Перед изменением схемы убедитесь, что текущая migration history применена к development database.

2. Добавьте поле description в Product

Откройте `prisma/schema.prisma` и добавьте nullable-поле `description`. Nullable-вариант позволяет сохранить существующие строки, в которых описание пока отсутствует.

`prisma/schema.prisma`

```
model Product {
  id          Int      @id @default(autoincrement())
  name        String   @db.VarChar(150)
  price       Int
  description  String?
  categoryId  Int       @map("category_id")
  inStock     Boolean   @default(true) @map("in_stock")
  category    Category @relation(fields: [categoryId], references: [id])

  @@map("products")
}
```

3. Создайте миграцию

PowerShell

```
npx prisma@7 migrate dev --name add-product-description
```

После выполнения команды в `prisma/migrations` должен появиться новый каталог с файлом `migration.sql`.

Ожидаемая структура

```
prisma/
├── schema.prisma
├── migrations/
│   └── YYYYMMDDHHMMSS_add-product-description/
│       └── migration.sql
└── seed.js
```

4. Изучите migration.sql

Откройте созданный `migration.sql`. Для добавления nullable-поля SQL должен соответствовать идее добавления нового столбца без удаления существующих строк.

Пример migration.sql

```
ALTER TABLE "products"
ADD COLUMN "description" TEXT;
```

Важно понимать

Prisma Migrate создаёт обычный SQL. Миграция является воспроизводимым шагом изменения PostgreSQL, а не отдельным форматом базы данных.

5. Сгенерируйте Prisma Client

PowerShell

```
npx prisma@7 generate
```

В Prisma ORM v7 после изменения схемы явно запускайте генерацию Prisma Client.

6. Проверьте новое поле через API

Измените создание Product так, чтобы API мог принимать `description`.

services/productService.js

```
export async function addProduct(data) {
  return prisma.product.create({
    data: {
      name: data.name,
      price: data.price,
      description: data.description ?? null,
    },
  });
}
```

```
    categoryId: data.categoryId,  
    inStock: data.inStock  
  }  
});  
}
```

POST /api/products

POST /api/products

Content-Type: application/json

```
{  
  "name": "Mechanical Keyboard",  
  "price": 32000,  
  "description": "Механическая клавиатура для учебного магазина",  
  "categoryId": 2,  
  "inStock": true  
}
```

Используйте существующий `categoryId` из вашей базы данных. Убедитесь, что новое поле сохраняется и возвращается API.

7. Настройте seed-команду

Откройте `prisma.config.ts` и добавьте команду запуска seed в объект `migrations`.

prisma.config.ts

```
import "dotenv/config";  
import { defineConfig, env } from "prisma/config";  
  
export default defineConfig({  
  schema: "prisma/schema.prisma",  
  migrations: {  
    path: "prisma/migrations",  
    seed: "node prisma/seed.js"  
  },  
  datasource: {  
    url: env("DATABASE_URL")  
  }  
});
```

8. Создайте seed.js

Создайте файл `prisma/seed.js`. Сначала добавим категории с помощью `upsert()`, чтобы повторный запуск не создавал дубликаты.

prisma/seed.js

```
import { prisma } from "../config/prisma.js";  
  
async function main() {  
  const laptops = await prisma.category.upsert({  
    where: { name: "Ноутбуки" },  
    update: {},  
    create: { name: "Ноутбуки" }  
  });  
};
```

```

const accessories = await prisma.category.upsert({
  where: { name: "Аксессуары" },
  update: {},
  create: { name: "Аксессуары" }
});

console.log({ laptops, accessories });
}

main()
  .then(async () => {
    await prisma.$disconnect();
  })
  .catch(async (error) => {
    console.error(error);
    await prisma.$disconnect();
    process.exit(1);
  });

```

Метод `upsert()` удобен для Category, потому что поле `name` объявлено уникальным.

9. Добавьте тестовые товары в seed

Поле `Product.name` в текущей модели не является уникальным, поэтому перед созданием тестового товара сначала проверим его наличие.

Фрагмент внутри `main()`

```

const existingLaptop = await prisma.product.findFirst({
  where: {
    name: "Laptop Pro"
  }
});

if (!existingLaptop) {
  await prisma.product.create({
    data: {
      name: "Laptop Pro",
      price: 420000,
      description: "Тестовый ноутбук",
      categoryId: laptops.id,
      inStock: true
    }
  });
}

const existingMouse = await prisma.product.findFirst({
  where: {
    name: "Wireless Mouse"
  }
});

if (!existingMouse) {
  await prisma.product.create({
    data: {
      name: "Wireless Mouse",
      price: 12000,
      description: "Тестовая беспроводная мышь",
      categoryId: accessories.id,
      inStock: true
    }
  });
}

```

```
});  
}
```

Повторяемый seed

Повторный запуск seed не должен бесконтрольно создавать одинаковые тестовые данные. Используйте `upsert()` для уникальных сущностей или предварительную проверку.

10. Запустите начальное заполнение

PowerShell

```
npx prisma@7 db seed
```

После выполнения команды проверьте, что категории и тестовые товары существуют в PostgreSQL.

11. Проверьте данные через Prisma Studio

PowerShell

```
npx prisma@7 studio
```

Откройте Category и Product и убедитесь, что seed создал ожидаемые данные и связи.

12. Проверьте повторный запуск seed

PowerShell

```
npx prisma@7 db seed  
npx prisma@7 db seed
```

После двух повторных запусков убедитесь, что категории и тестовые товары не размножились.

13. Проверьте историю миграций

PowerShell

```
npx prisma@7 migrate status
```

Убедитесь, что миграция `add-product-description` применена и отсутствуют pending migrations.

14. Самостоятельное задание

1. Добавьте в модель Category поле `slug String? @unique`.
2. Создайте отдельную миграцию с понятным именем.
3. Откройте новый `migration.sql` и определите, какой SQL будет выполнен.

4. Выполните `prisma generate`.
5. Обновите seed так, чтобы категории получали slug `laptops` и `accessories`.
6. Повторно выполните `prisma db seed`.
7. Проверьте через Prisma Studio значения `slug` и отсутствие дубликатов.

15. Дополнительная проверка восстановления development database

Удаление данных

Следующий шаг удаляет данные development database. Выполняйте его только после того, как убедились, что используете учебную базу.

PowerShell

```
npx prisma@7 migrate reset  
npx prisma@7 generate  
npx prisma@7 db seed
```

После выполнения последовательности структура должна восстановиться из migration history, а стартовые данные — из seed.

Требования к результату

- Поле `description` добавлено в Product через отдельную миграцию.
- Миграция хранится в `prisma/migrations`.
- Содержимое `migration.sql` изучено и соответствует изменению модели.
- `prisma migrate status` не показывает неприменённых миграций.
- После изменения схемы выполнен `prisma generate`.
- Product API работает с полем `description`.
- `prisma.config.ts` содержит команду `seed`.
- Файл `prisma/seed.js` создаёт стартовые категории и товары.
- Повторный seed не создаёт неконтролируемые дубликаты.
- Данные проверены через Prisma Studio.
- Самостоятельная миграция `Category.slug` выполнена отдельно.

Критерии оценивания

№	Критерий
1	Product.description добавлено в Prisma Schema
2	Создана отдельная migration add-product-description
3	migration.sql соответствует изменению схемы
4	Миграция успешно применена к PostgreSQL
5	Prisma Client повторно сгенерирован
6	API создаёт Product с description
7	Seed настроен через prisma.config.ts

№	Критерий
8	Category создаются воспроизводимо
9	Тестовые Product создаются без неконтролируемых дубликатов
10	prisma db seed выполняется без ошибок
11	Данные проверены через Prisma Studio
12	Студент различает назначение migration и seed
13	Студент различает migrate dev и migrate deploy
14	Самостоятельная миграция Category.slug выполнена
15	Студент может восстановить development database из migrations и seed